Java Abstraction

Hiding Complexity, Showing Essentials

codehive

What is Abstraction?

Data **abstraction** is the process of hiding certain details and showing only essential information to the user. It helps reduce programming complexity and effort.

In Java, abstraction is achieved using two main tools:

1. **Abstract Classes**
2. **Interfaces**

The "Abstract" Keyword

The `abstract` keyword is a non-access modifier, used for classes and methods. It acts as a blueprint for other classes to follow.

1. Abstract Classes & Methods

An **Abstract Class** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

An **Abstract Method** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

Java Syntax

```
abstract class Animal {
  // Abstract method (no body)
  public abstract void animalSound();

  // Regular method (has a body)
  public void sleep() {
    System.out.println("Zzz");
  }
}
```

⚠️ Critical Rule:

If you try to instantiate an abstract class directly, Java will throw an error:

```
Animal myObj = new Animal(); // ERROR: Cannot instantiate
```

2. Implementing Abstraction

To use the abstract class defined previously, we must **extend** it with a subclass. This subclass is responsible for filling in the "missing" logic of the abstract methods.

Pig.java

```
// Subclass (inherit from Animal)
class Pig extends Animal {
  public void animalSound() {
    // The body of animalSound() is provided here
    System.out.println("The pig says: wee wee");
  }
}
```

Putting It All Together

Now we can create a Main class to see how these two classes interact effectively.

3. Complete Code Example

Main.java

```
abstract class Animal {
  public abstract void animalSound();
  public void sleep() {
    System.out.println("Zzz");
  }
}

class Pig extends Animal {
  public void animalSound() {
    System.out.println("The pig says: wee wee");
  }
}

class Main {
  public static void main(String[] args) {
    Pig myPig = new Pig(); // Create a Pig object
    myPig.animalSound();   // Call abstract method
    myPig.sleep();         // Call inherited regular method
  }
}
```

Console Output:

The pig says: wee wee

Zzz

4. Why Use Abstraction?

Abstraction is a fundamental pillar of Object-Oriented Programming (OOP). Its benefits include:

- **Security:** It hides the internal implementation details and only reveals the relevant methods to the user.
- **Maintainability:** Changes to the internal code of the abstract class don't necessarily affect the users of the class as long as the interface remains the same.
- **Templates:** It allows you to define a common template for a group of related subclasses, ensuring they all implement certain methods.

> **Pro Tip:** Use an *Abstract Class* if you want to share code among several closely related classes. Use an *Interface* if you want to define a contract for classes that might not be related at all.

Summary Checklist

Keep these points in mind for your next Java project:

- ✓ **Abstract Class:** Cannot be instantiated.
- ✓ **Abstract Method:** Declared without a body.
- ✓ **Subclassing:** Use the `extends` keyword to implement abstract methods.
- ✓ **Hybrid Nature:** Abstract classes can contain both abstract and concrete (regular) methods.
- ✓ **Encapsulation:** Abstraction helps in hiding "how" things work and showing "what" they do.

Happy Coding!

codehive Learning Series